



Escalonamento de Tarefas

Prof. Pedro Botelho

Universidade Federal do Ceará

13 de abril de 2026

Conteúdo



Introdução ao Escalonamento

Escalonamento em Sistemas em Lote

Escalonamento em Sistemas Iterativos

Escalonamento em Sistemas de Tempo Real

Escalonamento de Threads

Conclusão



Introdução ao Escalonamento

Escalonamento em Sistemas em Lote

Escalonamento em Sistemas Iterativos

Escalonamento em Sistemas de Tempo Real

Escalonamento de Threads

Conclusão

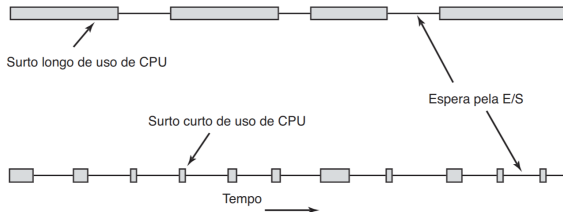


- ▶ Múltiplos processos
- ▶ Recurso compartilhado → CPU
- ▶ SO deve escolher sequência de execução
- ▶ Parte responsável no SO: **escalonador**
- ▶ Política de escolha: algoritmo de escalonamento
- ▶ Aplicação em processos ou threads (**tarefas**, no geral)



- ▶ Sistemas em lotes
 - ▶ Próxima tarefa existente na fita
- ▶ Sistemas multiprogramados
 - ▶ Usuários esperando serviço
 - ▶ Possível coexistência: lotes + usuários
 - ▶ Decisão sobre tipo de tarefa a executar
 - ▶ Desempenho percebido
- ▶ Computadores pessoais
 - ▶ Usuário monotarefa
 - ▶ CPU raramente um gargalo
 - ▶ Gargalo na entrada de dados
 - ▶ Escalonamento tem papel secundário

- ▶ Servidores/Estações de trabalho
 - ▶ Múltiplos processos competindo
 - ▶ Decisão influencia no desempenho percebido
 - ▶ Exemplo: Solicitações de usuários x Acumulação de estatísticas diárias
 - ▶ Eficiência no uso da CPU: custo de chaveamento
- ▶ Comportamento dos processos
 - ▶ Alternância entre CPU e E/S
 - ▶ CPU x I/O bounded
 - ▶ CPUs mais rápidas → Tendência a I/O bounded





- ▶ Tomada de decisão
 - ▶ Criação de um novo processo
 - ▶ Término de um processo
 - ▶ Bloqueio
 - ▶ Interrupção
- ▶ Tipos de escalonadores
 - ▶ Tratamento de Interrupção: Preemptivo/Não preemptivo
 - ▶ Ambiente: Lote/Interativo/Tempo Real

Objetivos de um Escalonador



- ▶ Gerais
 - ▶ Justiça — dar a cada processo uma porção justa da CPU
 - ▶ Aplicação da política — verificar se a política estabelecida é cumprida
 - ▶ Equilíbrio — manter ocupadas todas as partes do sistema
- ▶ Lote
 - ▶ Vazão (throughput) — maximizar o número de tarefas por hora
 - ▶ Tempo de retorno — minimizar o tempo entre a submissão e o término
 - ▶ Utilização de CPU — manter a CPU ocupada o tempo todo

Objetivos de um Escalonador



- ▶ Interativo
 - ▶ Tempo de resposta — responder rapidamente às requisições
 - ▶ Proporcionalidade — satisfazer às expectativas dos usuários
- ▶ Tempo Real
 - ▶ Cumprimento dos prazos — evitar a perda de dados
 - ▶ Previsibilidade — evitar a degradação da qualidade em sistemas multimídia



Introdução ao Escalonamento

Escalonamento em Sistemas em Lote

Escalonamento em Sistemas Iterativos

Escalonamento em Sistemas de Tempo Real

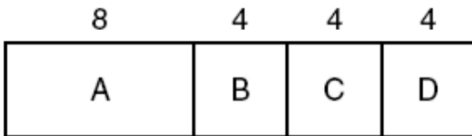
Escalonamento de Threads

Conclusão

First Come, First Served (FCFS)



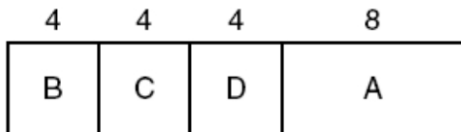
- ▶ Não preemptivo
- ▶ CPU alocada na ordem de requisição
- ▶ Novos processos para o final da fila
- ▶ Processo bloqueia → primeiro da fila executa
- ▶ Processo bloqueado para o final da fila



Shortest Job First (SJF)



- ▶ Não preemptivo
- ▶ Tempos de execução conhecidos
 - ▶ Adequado quando todos processos disponíveis



Shortest Remaining Time Next (SRTN)



- ▶ Versão preemptiva do SJF
- ▶ Tempos de execução conhecidos
- ▶ Menor tempo de execução restante → Maior prioridade



Introdução ao Escalonamento

Escalonamento em Sistemas em Lote

Escalonamento em Sistemas Iterativos

Escalonamento em Sistemas de Tempo Real

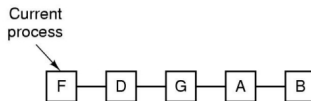
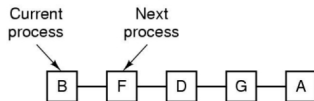
Escalonamento de Threads

Conclusão

Round Robin (RR)



- ▶ Chaveamento circular
- ▶ Preemptivo
- ▶ Cada processo tem um tempo para executar (*quantum*)
- ▶ Ao final do *quantum*, outro processo é executado
- ▶ Tamanho do *quantum* → Parâmetro de projeto
- ▶ Overhead na troca de contexto x Tempo de resposta
- ▶ Preferível minimizar preempção

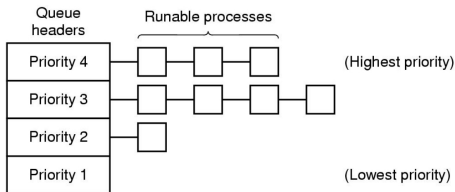


Escalonamento por Prioridades



- ▶ A cada processo é atribuída uma prioridade (estática ou dinâmica)
- ▶ O processo com maior prioridade é executado
- ▶ Possível problema: processos com alta prioridade não liberam CPU
- ▶ Possíveis soluções:
 - ▶ Redução gradativa de prioridade do processo em execução
 - ▶ *Quantum* máximo
- ▶ Prioridades dinâmicas para atingir objetivos
 - ▶ Processos orientados à E/S
 - ▶ Prioridade inverso da última fração de *quantum*

Filas Múltiplas



- ▶ CTSS: MIT utilizando IBM 7094
- ▶ Chaveamento de processo muito lento: um processo na memória
- ▶ Processos CPU bounded → *quantum* grande
- ▶ Classes de prioridade mais alta → um *quantum*
- ▶ Cada classe de menor prioridade → 2x *quantum*
- ▶ Processo preemptado → rebaixado de classe

Filas múltiplas (Exemplo)



- ▶ Processo com tempo de execução $100\times$ *quantum*
- ▶ Inicialmente com maior prioridade (1 *quantum*)
- ▶ Próximas execuções: 2, 8, 16, 32, 64 *quanta*
- ▶ 7 trocas de contexto (ao invés de 100)
- ▶ Diminuição de frequência de execução
 - ▶ Mais tempo para processos iterativos rápidos

Shortest Process First/Next (SPF/SPN)



- ▶ Similar ao aplicado a sistemas em lote
- ▶ Problema em conhecer a priori a duração dos processos
- ▶ Estimativa baseada em execuções passadas
$$T_{n+1} = aT_{n-1} + (1 - a)T_n$$
onde $a < 1$ é um fator de esquecimento
- ▶ Também chamada de *aging* (envelhecimento)

Escalonamento Garantido



- ▶ Garantia de desempenho
- ▶ Dados n processos, cada um deve ter $\frac{1}{n}$ da CPU
- ▶ Processo criado no instante $t_{i,start}$
- ▶ Processo tem direito de $t_{i,share} = \frac{t_{i,now} - t_{i,start}}{n}$
- ▶ Tempo efetivamente utilizado: $t_{i,used}$
- ▶ Fator de utilização $\frac{t_{i,used}}{t_{i,share}}$
- ▶ Quanto menor o fator, maior a prioridade

Escalonamento por Loteria



- ▶ Cada processo tem bilhetes de loteria
- ▶ A cada ponto de escalonamento, um bilhete é sorteado
- ▶ Prioridade implementada pelo número de bilhetes
- ▶ Cooperação pela troca de bilhetes

Escalonamento fração justa (*fair share*)



- ▶ Preocupação com a propriedade dos processos
- ▶ Alocação de CPU entre usuários
- ▶ Diversas políticas de distribuição entre usuários



Introdução ao Escalonamento

Escalonamento em Sistemas em Lote

Escalonamento em Sistemas Iterativos

Escalonamento em Sistemas de Tempo Real

Escalonamento de Threads

Conclusão

Escalonamento em Sistemas de Tempo Real



- ▶ Tempo tem uma função essencial
- ▶ Reação a estímulos externos dentro do prazo
- ▶ Duas principais categorias
 - ▶ Crítico
 - ▶ Não-crítico
- ▶ Escalonador deve tentar garantir atendimento a prazos
 - ▶ *Deadline*
- ▶ Escalonamento de tempo real
 - ▶ Estático/dinâmico
 - ▶ Online/Offline
- ▶ Processos (tasks) periódicos, aperiódicos e esporádicos
- ▶ Teste de escalonabilidade



Introdução ao Escalonamento

Escalonamento em Sistemas em Lote

Escalonamento em Sistemas Iterativos

Escalonamento em Sistemas de Tempo Real

Escalonamento de Threads

Conclusão

Escalonamento de Threads



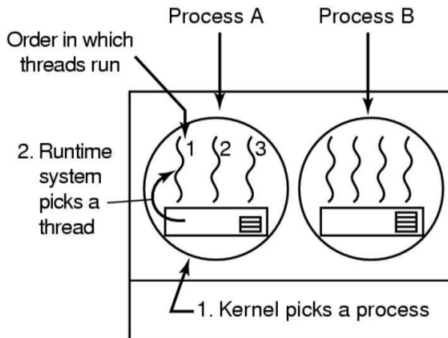
- ▶ Processos + Threads → Dois níveis de paralelismo
- ▶ Threads de usuário x Threads de núcleo
- ▶ Threads de usuário
 - ▶ Chaveamento usa poucas instruções
 - ▶ Mais rápido
 - ▶ Espera de E/S bloqueia todo processo
 - ▶ Possível escalonador específico
 - ▶ Exemplo: Servidor Web
 - ▶ Despachante + Operários
 - ▶ Alternância de execução
 - ▶ Maximização de paralelismo
- ▶ Threads de núcleo
 - ▶ Chaveamento completo de contexto
 - ▶ Mais lento
 - ▶ Espera de E/S bloqueia apenas thread
 - ▶ Núcleo pode usar informação da relação Processo ↔ Threads

Escalonamento de Threads de Usuário



- ▶ Núcleo não tem ciência das threads
- ▶ Escalonamento → Escolha de processos
- ▶ Escalonador de thread dentro do processo
- ▶ Interrupção de relógio age no processo
- ▶ Exemplo
 - ▶ Processo A com threads A1, A2 e A3
 - ▶ Escalonador escolhe processo A para executar
 - ▶ Escalonador de thread escolhe thread A1 para executar
 - ▶ Thread A1 executa até término, auto-bloqueio ou troca de processo
 - ▶ *Quantum* de 50ms e surto de CPU de 5ms (cada thread)
 - ▶ Sequência: A1-A2-A3-A1-A2-A3-A1-A2-A3-A1 durante *quantum*

Exemplo: Escalonamento de Threads de Usuário



Possible: A1, A2, A3, A1, A2, A3

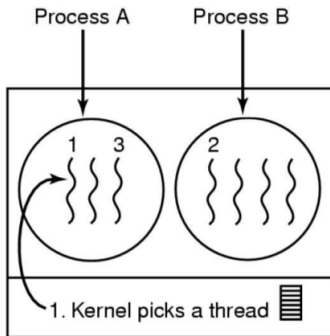
Not possible: A1, B1, A2, B2, A3, B3

Escalonamento de Threads de Núcleo



- ▶ Núcleo escolhe thread a ser executada
- ▶ Pode ou não considerar relação Processo $\leftrightarrow$ Threads
- ▶ Interrupção de relógio age nas threads
- ▶ Exemplo
 - ▶ Processo A com threads A1, A2 e A3
 - ▶ Processo B com threads B1, B2, B3 e B4
 - ▶ Escalonador escolhe thread A1 para executar
 - ▶ Thread A1 executa até término, auto-bloqueio ou Interrupção (escalonador)
 - ▶ Sequência de threads depende da política de escalonamento

Exemplo: Escalonamento de Threads de Núcleo



Possible: A1, A2, A3, A1, A2, A3

Also possible: A1, B1, A2, B2, A3, B3

Conteúdo



Introdução ao Escalonamento

Escalonamento em Sistemas em Lote

Escalonamento em Sistemas Iterativos

Escalonamento em Sistemas de Tempo Real

Escalonamento de Threads

Conclusão

Dúvidas?



Obrigado pela atenção!